



Tecnológico de Monterrey

Campus Querétaro

Documentación

Valeria Escalante Campos A01709023
Lucca Traslosheros Abascal A01713944
Ricardo Rodríguez Medina A01710266
José Miguel Ramírez Gutiérrez A01712628

Pedro Nájera García

Implementación de internet de las cosas
502

5 de Diciembre del 2025

Introducción

DetectaGas es un sistema IoT diseñado para detectar fugas de gas LP en hogares y enviar alertas inmediatas tanto localmente con una alarma sonora, como remotamente con una notificación a sus dispositivos. Utiliza un sensor MQ-5 y un módulo ESP32 conectado a internet mediante WiFi, enviando los datos a un backend propio y a una interfaz web de monitoreo en tiempo real.

Este proyecto ayuda a prevenir accidentes como incendios, explosiones o intoxicaciones, especialmente en hogares donde el gas LP es un riesgo constante por instalaciones defectuosas o fugas inadvertidas.

Objetivo del Proyecto

Diseñar un sistema IoT capaz de detectar niveles peligrosos de gas LP, activar una alarma local y enviar alertas remotas mediante Internet, permitiendo al usuario supervisar el estado de su hogar desde cualquier lugar.

Público Objetivo

- Familias que utilizan gas LP en su hogar.
- Viviendas con boiler, estufa, secadora, calentadores o tanques estacionarios.
- Personas que pasan mucho tiempo fuera de casa y necesitan monitoreo remoto confiable.
- Gente de la tercera edad que requiere monitoreo por familiares.

Componentes del Sistema

Hardware

- ESP32: Microcontrolador con WiFi integrado, encargado de leer el sensor y enviar datos.
- Sensor MQ-5: Detecta gases inflamables como butano, propano y vapores combustibles.
- LED Verde: Indica funcionamiento correcto del sistema.
- LED Rojo: Indica error o peligro.
- Buzzer: Alarma sonora.
- Protoboard: Para montaje del prototipo.
- Cables jumper: Conexiones internas del circuito.
- Fuente con pilas AA: Proporcionar corriente de 5V en manera portátil.

Diagrama de Funcionamiento

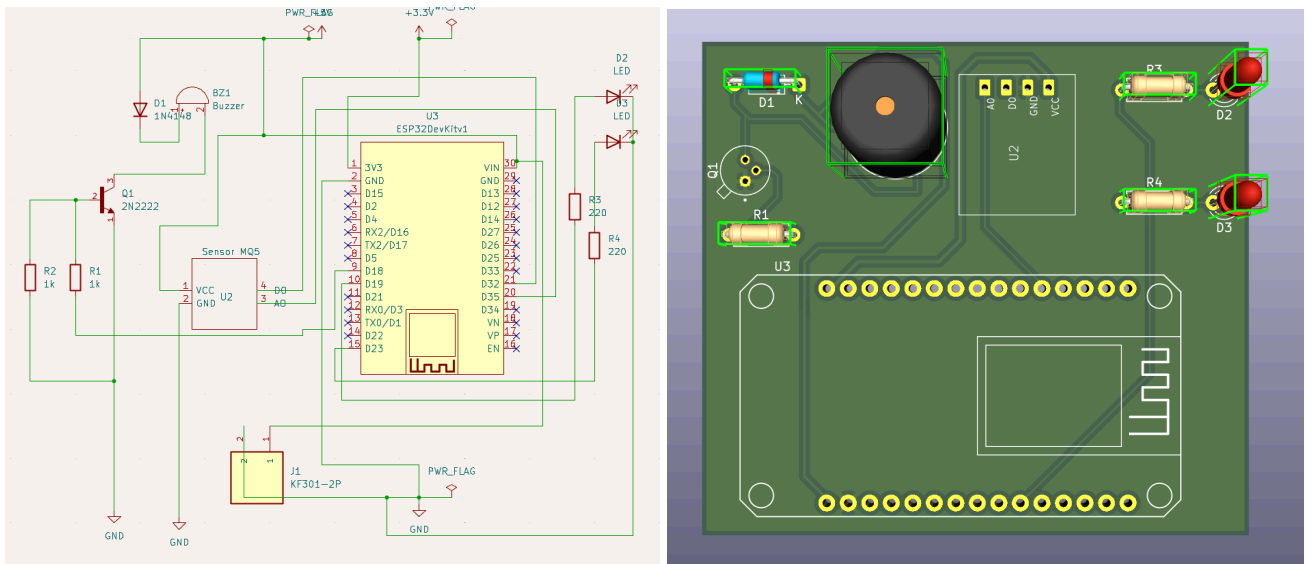
MQ-5 → ESP32 → MQTT Broker (Eclipse Mosquitto) → Backend (Rust Axum) → MySQL + WebSocket → Frontend Vite React

1. El MQ-5 mide la concentración de gas.
2. El ESP32 procesa la lectura.
3. Pública la lectura al broker MQTT.
4. El backend recibe los datos, los guarda en MySQL y los retransmite por SSE o Websockets.
5. El frontend obtiene las lecturas en tiempo real y muestra alertas.
6. Si el nivel es mayor o igual a 300 ppm, el dispositivo activa la alarma local y envía notificación.

Conexiones

- MQ-5: Pin analógico de ESP32 GPIO 32.
- LED verde: GPIO 15 con resistencia 220Ω.
- LED rojo: GPIO 19 con resistencia 220Ω.
- GND comunes.
- VCC 5.5V para sensor y buzzer; y 3.3V LEDs.

Esquema eléctrico y PCB



Software necesario

- Se utilizó mpremote y esptool como utilidades para la comunicación entre la computadora y el ESP32.
- MySQL versión 9.5
- Rust versión 1.91.1
 - Lenguaje para el backend.

- Vite configurado para usar TS-SWC-REACT.
 - Leanguaje y framework para el frontend

Módulos del Backend

Nuestro backend se compone de los siguientes modulos.

- `iot-cli`: es la interfaz para inicializar el programa.
- `iot-db`: Conexión y control con la base de datos.
- `iot-entities`: Los modelos y representación de servicios que se utilizan para representar la información.
- `iot-listener`: Conexión con el broker mqtt.
- `iot-server`: El servidor HTTP que sirve al frontend o clientes que necesiten la información y alertas.

Procedimiento para configuración

- Configuración del ESP32 con esptool
- Importar librerías de micropython.
- Cargar el firmware correspondiente.
- Configurar credenciales WiFi dentro del código:
 - `SSID = "tuWifi"`
 - `PASS = "tuPassword"`
- Funcionamiento del Sensor MQ-5
- Definir un ID único específico para el sensor.

Umbral de detección

- El sistema utiliza un umbral de: 300 ppm
- Si la lectura es mayor o igual a 300 ppm:
 - Se enciende el LED rojo.
 - Se activa la alarma sonora.
 - Se envía alerta remota.
- Si la lectura es menor a 300 ppm:
 - LED verde encendido.
 - Sistema estable.

Configuración del Broker

Broker: Eclipse Mosquitto

Para el entorno de pruebas se utilizó la configuración default pero activando clientes anónimos. También se usó el puerto default que es el **1883**.

Topic usado:

- `iot/sensor/gas`

Endpoints del servidor HTTP

- `WS /sensor/ws`: WebSocket tiempo real
- `GET /sensor/live`: SSE tiempo real
- `GET /sensor/`: Obtener todos los eventos
- `GET /sensor/<id>`: Obtener información de un sensor
- `GET /sensor/<id>/history/`: Obtener historial de un sensor
- `POST /ping`: Estado actual del sistema

Características del Backend

- Página web accesible desde cualquier dispositivo.
- Gráficas en tiempo real mediante WebSocket.
- Indicador de estado: “Seguro” o “Riesgo elevado - Fuga detectada”.
- Historial descargable.

Modo de Uso para el Usuario Final

- Conectar el dispositivo a energía con 4 baterías AA de 1.5V.
- Esperar 3 segundos a que los LEDs indiquen funcionamiento correcto.
- Abrir la página web en el celular o computadora.
- Supervisar los niveles de gas en tiempo real.
- Si ocurre una fuga recibirá alerta en tiempo real.
- Sustentabilidad (ODS)
 - Evita accidentes - ODS 11 (Ciudades sostenibles)
 - Seguridad doméstica - ODS 3 (Salud y bienestar)
 - Tecnología accesible - ODS 9 (Industria e innovación)

Innovación

DetectaGas es innovador por:

- Integrar IoT + redes MQTT + WebSockets en un sistema doméstico accesible.
- Proveer supervisión remota en tiempo real.
- Combinar backend personalizado en Rust para máxima eficiencia.
- Ofrecer una interfaz limpia, moderna y multiplataforma.

Los experimentos realizados validaron:

- Funcionamiento estable del MQ-5.
- Alerta confiable a partir de 300 ppm o más.
- Latencia mínima entre detección y notificación (menor a 1 segundo).
- Almacenamiento correcto del historial en MySQL.

Conclusiones

DetectaGas es un proyecto IoT completo, funcional e innovador, que combina electrónica, programación embebida, bases de datos, backend robusto y una plataforma web moderna. Demuestra habilidades de ingeniería, análisis, sustentabilidad e innovación, cumpliendo totalmente con los criterios de la evidencia.

Código Fuente

<https://github.com/DevSlashRichie/iot-frontend>

<https://github.com/DevSlashRichie/iot-backend>

Referencias

R. A. Light, "Mosquitto: server and client implementation of the MQTT protocol," The Journal of Open Source Software, vol. 2, no. 13, May 2017, DOI: 10.21105/joss.00265